

Stage 4: Training with Brax PPO

important parameters for PPO (Proximal Policy Optimization)

- # of parallel environments
- rollout/unroll length
- minibatching
- updates per batch
- discounting
- entropy/exploration
- learning rate
- observation normalization
- total environment

Steps: Train (Environment + Brax PPO) → trained neural net

Export (JAX/Brax parameters → portable .npz file

Reimplement in numpy (.npz + observation → 12 joint actions)

PPO uses 2 neural nets, policy and value

policy: given what I see, what should I do?

value: given the current situation, how much future reward should I expect

only policy gets deployed on robot, value helps with training

Part 1: `pup/train/train_ppo.py`

connecting Pup joystick environment
 Brux's PPO implementation
 network architecture
 domain randomization
 logging/checkpoints

Part 2: brief smoke test to check if pipeline works

Part 3: Cclab run

Part 4: export trained policy

45 observations



hidden layer 512



hidden layer 256



hidden layer 128



output

each layer contains

kernel = weights } saved to policy.npz
bias = biases } (around 200 kb)

Brax normalizes 45 observations

$$\frac{\text{raw obs} - \text{mean}}{\text{std}} = \text{norm obs}$$

Part 5: rewrite inference in pure NumPy

implementing forward pass of policy in NumPy

$X = X @ \text{kernel} + \text{bias}$ for each layer

swish applied for hidden layers ($x = x \cdot \text{sigmoid}(x)$)

final layer is linear

24 number output { first 12: action mean
second 12: action standard deviation

PPO wants a distribution so it can explore during training

deterministic behavior wanted for deployment

↳ take first 12 = mean → calculate action = $\tanh(\text{mean})$

ignoring standard deviation outputs

convert actions to actual joint targets

Numpy tests compare Brax/JAX policy to Numpy policy
to detect any significant differences (greater than 0.0001)
across 256 random observations